MyDAPLink_F103 使用手册

把一块 STM32F103C8T6 烧进两个固件，变成一台 CMSIS-DAP 烧录器 —— 从零到能用的完整步骤

源码：ARM 官方 DAPLink　目标接口：PB6 / PB7 / PB8 / PB9　已本机编译 · 3 个固件

一句话：烧 `MyDAPLink_BL.hex`（到 `0x08000000`）+ `MyDAPLink_IF.hex`（到 `0x0800C000`），这块 F103 就成了一台烧录器，可以用它给别的 ARM Cortex-M 芯片烧录 / 调试。

⚠️ 拿到这个包的其他人请注意

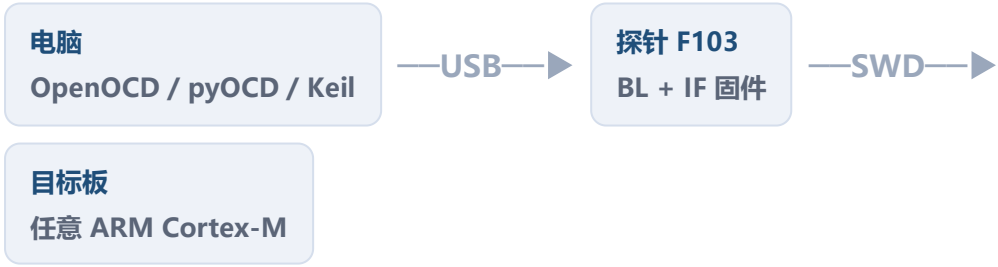
- 本包**可搬移**——整个文件夹拷到任意盘符、任意目录都能用。
- `COM3`（第 9 节、`rescue/isp_rescue.py`）→ 换成**你电脑上 USB-TTL 实际的 COM 口。**
- `E:/LEDTEST.elf`（在 `tools/verify_flash.gdb` 里）和第 6.1 节的 `<目标程序.elf>` → 换成**你自己目标程序**的 `.elf` 路径，那是你自己的文件，不在本包里。
- `source tools/verify_flash.gdb`（第 6.1 节）→ 建议改成**绝对路径。**
- 需要你**自己安装**：`openocd`、`arm-none-eabi-gdb`，并且加入 `PATH`（自检：`openocd --version` 能打印版本号）。
- 手册里所有 `out/xxx.hex`、`tools/xxx`、`rescue/xxx` 都是**相对路径**—— 请先 `cd` 到本包根目录再执行命令，否则会报 `No such file or directory`。
- 只有想**重新编译**时才需要 `arm-none-eabi-gcc`、Python、GNU make，而且必须先按 `README.md` 第 12 节把 `mbed-os` 补回来（本精简版已移除，包里不含）。

目录

1. 这是什么
2. 准备工作
3. 第一步：把固件烧进探针
4. 第二步：验证探针就绪
5. 第三步：接到目标板
6. 第四步：烧录 / 调试目标
7. 日常：升级与拖拽烧录
8. 常见问题排查
9. 刷坏了怎么救
10. 固件清单
11. 能烧哪些芯片 / 为什么有的 ARM 不支持

1. 这是什么

把一颗普通 STM32F103 变成「电脑 ↔ 目标芯片」之间的桥：



变成探针后，它通过 USB 向电脑提供：

USB 接口	用途
CMSIS-DAP v2	调试 / 烧录主通道，给 OpenOCD、pyOCD、Keil、IAR 用
CDC（虚拟串口）	多出一个 COM 口，可当串口调试
MSC（大容量存储）	多出一个「U 盘」，拖 hex/bin 进去即烧（需 MSC 版固件）
HID	部分免驱场景

向目标芯片走的是 **SWD** 协议（SWCLK + SWDIO 两根信号线）。

2. 准备工作

物品	说明
探针芯片	一块 STM32F103C8T6，Flash 必须 **128KB**（多数 Blue Pill 是 128KB）
ST-Link（首次用）	只用来第一次把固件烧进探针，烧完就不需要它了
杜邦线	建议短一些（<15cm），SWD 对线长敏感
目标板	任意要调试的 ARM Cortex-M 芯片 —— 完整支持范围与限制见 **11**
电脑软件	`openocd`、`arm-none-eabi-gdb`（可选 pyOCD / Keil / IAR）

> **先确认芯片是 128KB。** 64KB 的板子装不下固件。用 ST-Link 查：
>
> ```
> st-info --probe      # 看 "flash:" 一行，应为 128k
> ```

> ⚠️ **动手前先自检工具链。** 上表里的 `openocd`、`arm-none-eabi-gdb` 需要你 **自己安装并加入 PATH**；`st-info` 属于 `stlink-tools`，也要另外装。 在命令行敲一句 `openocd --version`，能打印出版本号才可以继续。

3. 第一步：把固件烧进探针

此刻它还只是一块**普通 F103**，必须用外部调试器（ST-Link）烧。 要烧 `out/` 里两个文件：**先 BL，后 IF。**

3.1 接线（ST-Link → 探针芯片）

ST-Link	探针 F103
SWCLK	PA14
SWDIO	PA13
GND	GND
3.3V	3.3V

3.2 烧录（三种方式任选）

方式 A • OpenOCD（推荐，已实测）

```
# hex 自带地址，命令里不要再写地址！
openocd -f interface/stlink.cfg \
  -f target/stm32f1x.cfg \
  -c "init; reset halt" \
  -c "flash write_image erase \
      out/MyDAPLink_BL.hex" \
  -c "flash write_image erase \
      out/MyDAPLink_IF.hex" \
  -c "reset halt" \
  -c "verify_image out/MyDAPLink_BL.hex" \
  -c "verify_image out/MyDAPLink_IF.hex" \
  -c "reset run; shutdown"
```

成功标志：`wrote 48128 bytes` + `wrote 80896 bytes` + 两行 `verified ... bytes`。

方式 B • st-flash

```
# 用 .bin 时要显式给地址
st-flash --reset write \
  out/MyDAPLink_BL.bin 0x08000000
st-flash --reset write \
  out/MyDAPLink_IF.bin 0x0800C000

# 或用 .hex（读内部地址，无需再给）
```

> ⚠️ **先确认两件事：** ① 下方命令里的 `out/MyDAPLink_BL.hex` 是**相对路径**，请在**包根目录**下执行，或改成固件的绝对路径； ② `-f interface/stlink.cfg` 是给 **ST-Link** 用的，若你用的是 J-Link 或别的 CMSIS-DAP 调试器，要换成对应的 interface 配置文件。

> ⚠️ **必看的坑（已实际踩过）：** 给 `.hex` 文件烧录时**千万不要再传地址。** 若写成 `flash write_image erase out/MyDAPLink_BL.hex 0x08000000`， OpenOCD 会把 `0x08000000` 当成「偏移量」**再叠加一次**，地址翻倍成 `0x10000000`，于是报 `no flash bank found for address 0x10000000` 且 `wrote 0 bytes` ——**一个字节都没写进去**， 但日志前面照样打印 `** Programming Finished **`，极易误判为成功。

方式 C • STM32CubeProgrammer： 图形界面里依次打开 BL / IF 两个 hex， 确认起始地址（BL=`0x08000000`，IF=`0x0800C000`）后烧录。

> **想要拖拽 U 盘?** 把第二步的 `MyDAPLink_IF.hex` 换成 `MyDAPLink_IF_MSC.hex` （地址同样 `0x0800C000` ）。两者只能选一个。

3.3 烧完

1. **拔掉 ST-Link**；
2. 把探针芯片的 USB 口插到电脑（Blue Pill 上是 PA11/PA12 那个口）；
3. 进入下一步验证。

4. 第二步：验证探针就绪

插入 USB 后，Windows 里应多出下列设备（`Status` 全为 `OK`）：

```
Get-PnpDevice -PresentOnly | Where-Object { $_.InstanceId -like '*VID_0D28*' } |
  Format-Table Status, FriendlyName, InstanceId -AutoSize
```

FriendlyName	说明
USB Composite Device	复合设备总入口
CMSIS-DAP v2	调试通道 **关键**
USB 串行设备 (COMx)	虚拟串口
USB 输入设备	HID
可移动磁盘	仅 MSC 版固件才有

再用 OpenOCD 空连一次，确认探针活着（**目标板先不接**）：

```
openocd -f interface/cmsis-dap.cfg -f target/stm32f1x.cfg -c "adapter speed 100"
```

> 探针正常时应看到：
>
> ```
> Info : CMSIS-DAP: FW Version = 2.1.0
> Info : CMSIS-DAP: Interface ready
> Info : clock speed 100 kHz
> ```
>
> 之后因为**目标板没接**会报 `cannot read IDR` 一类错误 —— **这是正常的**，说明探针已就绪、在等目标板。按 `Ctrl+C` 退出即可。

5. 第三步：接到目标板

探针的 SWD 目标接口固定在本设计的 **PB6 ~ PB9**（连续四脚，可做一排 4P 排针整体对插）。

#	探针	目标板	作用	必接?
1	**PB6**	SWCLK（STM32 上是 PA14）	时钟	✅ 必接
2	**PB7**	SWDIO（STM32 上是 PA13）	数据（单脚双向）	✅ 必接
3	**GND**	GND	共地	✅ 必接
4	**PB8**	NRST	复位	⚠️ 见下
5	3.3V	VCC	供电	⚠️ 见下
—	**PB9**	—	POWER_EN（可控供电）	❌ 默认不用

> ① **供电：** `PB9 = POWER_EN` 默认输出低 = **不向目标供电。** 目标板必须**自己单独上电**；也可以把探针 3.3V 接到目标 VCC 给它供电（注意电流够不够）。

> ② **NRST 通常不是必需的：**多数板接 PB6/PB7/GND 三根就能用。 只有个别目标板需要补上 PB8→NRST 才能进调试态。

SWD 速度：从 500 kHz 起，不要一上来就 1 MHz 实测坑

这是本设计**实测踩过的最大的坑**，优先级最高：

SWD 速度	实测结果
2000 / 1000 kHz	❌ `Failed to read memory at 0xe000ed00`（DP 握手成功、AP 读不到）
500 kHz	⚠️ 时好时坏，不可靠
250 kHz	✅ 稳定成功
100 kHz	✅ 稳定成功

为什么：杜邦线较长、接触电阻大、无端接时，SWD 波形在高速下会失真。 DP 握手只是简单一问一答、容忍度高；AP 读内存是连续多字节传输、对信号完整性要求高得多。 于是表现为「**DP 通、AP 不通**」—— 看着像接线或固件问题，其实是速度问题。

> **做法**：OpenOCD 一律用 `-c "adapter speed 100"` 起步（最稳），确认通了再往上试 250 / 500。换更短的杜邦线（< 15 cm）或直接焊接排线可显著提高可用速度。出现 `Failed to read memory` 时**先降速**，不要先去怀疑接线或固件。

6. 第四步：烧录 / 调试目标板

6.1 OpenOCD + GDB（通用，推荐）

终端 1 — 起 OpenOCD 服务（保持运行）：

```
openocd -f interface/cmsis-dap.cfg -f target/stm32f1x.cfg -c "adapter speed 100"
```

> 看到下面这两行，说明**目标板被真正识别**：
>
> ```
> Info : [stm32f1x.cpu] target has 6 breakpoints, 4 watchpoints   ← 关键
> Info : [stm32f1x.cpu] Examination succeed
> ```
>
> 若只看到 `starting gdb server on 3333`，而前面是 `Examination failed` 或 `Failed to read memory` → **先把速度降到 100 kHz 重试**，仍不行再查接线。

终端 2 — 开 GDB 并逐行敲：

```
arm-none-eabi-gdb <目标程序.elf>

# 进入 GDB 后：
target extended-remote localhost:3333
monitor reset halt
load                # 把 elf 烧进目标 Flash
compare-sections    # 比对 Flash 与 elf 是否一致
continue            # 跑起来
```

> GDB 里**不要**粘贴带 `(gdb)` 前缀或 `#` 注释的行，会报 `Undefined command`。要么一行行敲，要么用 `source 脚本.gdb`。用包内脚本：`(gdb) source tools/verify_flash.gdb`（只读自检）。

> ⚠️ **这两处路径要换成你自己的：**
>
> - `<目标程序.elf>` —— 换成**你自己**要烧的目标程序，它不在本包里；路径含空格记得加引号。
> - `tools/verify_flash.gdb` —— 这是**相对路径**。GDB 是按**你启动 GDB 时所在的目录**去找它的，所以要么先 `cd` 到包根目录再启动 GDB，要么直接写绝对路径，例如 `(gdb) source D:/MyDAPLink_F103_精简版/tools/verify_flash.gdb`。
>
> 该脚本开头还写着作者的测试程序路径，第一次用之前建议打开文件看一眼（见文件顶部注释）。

6.2 pyOCD

```
pyocd list                                # 确认探针在内
pyocd flash -t stm32f103c8 <你的.hex>     # 烧录
pyocd commander -t stm32f103c8            # 进交互调试
```

6.3 Keil / IAR

- **Keil：** `Options → Debug` 选 **CMSIS-DAP Debugger**；
- **IAR：** `Project → Options → Debugger` 选 **CMSIS DAP**。

6.4 拖拽烧录（仅 MSC 版固件）

1. 插上探针，打开多出来的「U 盘」；
2. 把目标的 `.hex` / `.bin` **拖进去**；
3. 探针自动烧录，完成后自动重新枚举。

拖拽烧录是**离线**的——探针自己完成，不经过电脑上的工具。

为什么有的版本没有这个盘、以及怎么切换 —— 见 **7.4**。

7. 日常：升级与拖拽烧录

7.1 只升级 IF（最常见）

BL 只烧一次，之后升级只需换 IF：

```
# .bin 给地址；.hex 自带地址、不要给
st-flash --reset write out/MyDAPLink_IF.bin 0x0800C000
```

或：MSC 版固件下，把新 IF 的 hex 拖进虚拟 U 盘即可，不必接 ST-Link。

7.2 切回 Bootloader（magic 文件）

`out/magic/START_BL.ACT` 是给 **MSC 版**用的：拖进虚拟 U 盘， 探针会重启进入 BL 模式，方便后续升级/恢复。它的内容就是两个字节 `BL` 。

7.3 Flash 布局（128KB 片）

区域	起始地址	大小
Bootloader (BL)	`0x08000000`	0xBC00 (48128 B)
Interface (IF)	`0x0800C000`	0x13C00 (80896 B)
Config (用户)	`0x0801FC00`	0x400

7.4 虚拟 U 盘（MSC）什么条件才会出现 常被问

> **一句话：不是所有 IF 固件都带虚拟盘 —— 只有「内嵌了目标 flash 算法」的那一版才会出现。**

固件	对应工程	内嵌 flash 算法	有虚拟 U 盘吗
`MyDAPLink_IF.hex`	`stm32f103xb_if`	❌ 无（all-family）	**没有**
`MyDAPLink_IF_MSC.hex`	`stm32f103xb_stm32f103rb_if`	✅ 有（STM32F103）	**有**

为什么：DAPLink 官方文档 `docs/USERS-GUIDE.md` 写明，MSD / 拖拽功能是 「构建里包含目标 flash 算法」时**自动启用**的；那些 all-family 构建本身没有绑定具体目标， 就**不带 MSD 盘** —— 它仍然能用 CMSIS-DAP 协议给任意芯片烧录 / 调试，只是没有那个拖拽盘。

对照本包两个工程的声明（`src/upstream/projects.yaml`）：

```
stm32f103xb_if:                         # → MyDAPLink_IF.hex
    - *module_if
    - records/family/all_family.yaml  # 只有各家 target_reset_*.c，无 flash 算法
    - records/usb/usb-bulk.yaml

stm32f103xb_stm32f103rb_if:             # → MyDAPLink_IF_MSC.hex
    - *module_if
    - records/board/stm32f103rb.yaml  # → 引入 stm32f103rb 的 flash 算法
```

- `records/family/all_family.yaml` 里只有各家的 `target_reset_*.c`（复位辅助代码），**一条 flash 算法都没有**；
- `source/family/st/stm32f103rb/target.c` 里有 `.flash_regions[0].flash_algo = (program_target_t *)&flash;`（配 `flash_blob.c`），**内嵌了 STM32F103 的烧录算法** → MSD 自动挂出。

> **不是「通用版把 MSC 代码裁掉了」。**两个 IF 版本**都**编了 MSC 的 USB 端点（`module_if` 里本身就含 `records/usb/usb-msc.yaml`，定义 `MSC_ENDPOINT` → `USBD_MSC_ENABLE=1`），差别只在「有没有内嵌目标 flash 算法」这一条。

容量：两版 IF 的 `.bin` 都是 80896 B（占用同一块 ROM 区 `0x0800C000`），**虚拟盘不额外占 Flash**；128 KB 的片放得下（BL + IF ≈ 123 KB）。

怎么切换：若当前芯片里是通用版 IF，想有虚拟盘 —— 用 ST-Link 把 IF 区擦掉重烧 MSC 版（**BL 不用动**）：

```
openocd -f interface/stlink.cfg -f target/stm32f1x.cfg \
  -c "init; reset halt" \
  -c "flash write_image erase out/MyDAPLink_IF_MSC.hex" \
  -c "reset run; shutdown"
```

拖进去的文件分别是什么：

拖入的文件	效果
目标的 `.hex` / `.bin`	烧到**目标芯片**（不是探针自己）；成功变成完成页面，失败生成 `FAIL.TXT`
`out/magic/START_BL.ACT`	探针**自己**重启进 BL，等待升级 IF
`msd_off.act` / `msd_on.act`	关掉 / 开回虚拟盘

拖入的文件	效果
`page_on.act` / `page_off.act`	临时允许页编程+扇区擦除 / 恢复

`.act` 命令全集见官方 `docs/MSD_COMMANDS.md`。

> **限制：**虚拟盘**只认它内嵌算法的那个目标系列**（本包这版是 STM32F103）。 要烧别的芯片（Nordic、Freescale 等），请走 **CMSIS-DAP 调试口**（OpenOCD / pyOCD）—— 那是通用协议，芯片无关，任何 ARM Cortex-M 都能烧。 完整的支持范围、以及「为什么有的 ARM 不支持」，见 **11**。

7.5 什么时候还需要 ST-Link? 常被问

把「烧录」分成四种，答案完全不一样：

你要烧的东西	需要外部调试器吗	怎么做
目标板固件（LEDTEST 之类）	❌ 不需要	探针的 CMSIS-DAP 口（OpenOCD / pyOCD），或 MSC 版拖拽虚拟盘
探针的 IF（当前 IF 带虚拟盘）	❌ 不需要	拖 `START_BL.ACT` 进 `DAPLINK` 盘 → 变 `MAINTENANCE` 维护盘 → 拖新 IF hex
探针的 IF（当前 IF 不带虚拟盘，如通用版）	✅ 需要	外部调试器（或另一块 DAPLink）
探针的 BL（首刷 / 换 BL / 救砖）	✅ 需要	外部调试器；最坏还有串口 ISP（见 **9**）

> **一句话：**日常迭代（改时钟、改引脚、换 target 工程 → 重编 IF）已经可以彻底拔掉 ST-Link； 只有动 BL、或者 IF 卡死到挂不出盘，才需要它。

为什么维护盘救不了 BL —— 三条源码级硬证据：

1. **维护盘只映射 IF 那一段。** `source/board/stm32f103xb_bl.c:42-43`：

   ```
   .flash_regions[0].start = 0x08000000 + KB(48),    // 0x0800C000
   .flash_regions[0].end   = 0x08000000 + KB(128),   // 0x08020000
   ```

   BL 自己那 48 KB（`0x08000000-0x0800C000`）**根本不在维护盘映射的地址范围里。**

2. **官方的「BL 自更新」机制本构建没启用。** `bootloader_update.c:39-41`：

```
#if !defined(DAPLINK_BOOTLOADER_UPDATE)
    #define DAPLINK_BOOTLOADER_UPDATE  0
#endif
```

该宏**全树只有这一处出现**，没有任何工程定义它 → 内嵌镜像 `image_size = 0`，`bootloader_check_and_update()` 退化成空操作。

3. **就算启用了也没地方放。** BL 镜像 48128 B，而 IF 区只有 `0x13C00 = 80896 B`， 实测占用 `text 61952 + data 18944 = 80896 B` —— **恰好占满、零余量**， 塞不进 48 KB 的 BL。

那「不用 ST-Link」的边界在哪? 关键在于有没有办法让探针停在维护盘。 BL 的跳出条件（`main_bootloader.c:275-278`）是**四个条件全真**才跳进 IF：

```
if (!reset_button_pressed()                          // 本板无专用复位键，恒为 false
        && g_board_info.target_cfg
        && validate_bin_nvic(IF 起始地址)             // IF 的向量表有效…
        && !config_ram_get_initial_hold_in_bl()) {   // …且没有被要求「停在 BL」
    /* 跳进 IF */
}
/* 否则留在 BL，挂出 MAINTENANCE 盘 */
```

于是只有三种局面：

局面	能进维护盘吗	结果
IF 正常，且带虚拟盘	✅	拖 `START_BL.ACT` 置 hold-in-bl 标志 → 重启停在维护盘
IF 被擦掉 / 向量表非法	✅	重启**自动**停在维护盘 —— **IF 刷坏也能自救**
IF 能跳转但卡死在 USB 初始化前	❌	既没有盘、又进不了维护盘 → 只能上外部调试器

> **第三行最容易踩：**引脚配错、时钟配错导致 IF 跑飞时，虚拟盘根本不会出现， 这时维护盘也帮不了你。好在 USB 没起来的 IF 一般也不会破坏 BL。

而且「外部调试器」不必非是 ST-Link。任何能当 SWD 主机的都行 —— **第二块 DAPLink 探针**最顺手（正好是「用 DAPLink 修 DAPLink」）： 接本探针的 `PB6(SWCLK) / PB7(SWDIO) / PB8(nRESET) / GND`，OpenOCD 换成 `-f interface/cmsis-dap.cfg` 即可。这条路一直通，是因为 `gpio.c` 里特意用了 `__HAL_AFIO_REMAP_SWJ_NOJTAG()` 保留探针芯片自己的 SWD 口 （官方默认

`SWJ_DISABLE` 会把探针自己的 PA13/PA14 关掉，那才真的会变砖）。 最坏的情况还有**串口 ISP**（BOOT0=1 + PA9/PA10），见 **9**。

8. 常见问题排查

现象	原因 / 解决
`cannot read IDR`	DP 层都不通。① 目标板没上电；② SWDIO(PB7) 接触不良/接反；③ SWCLK/SWDIO 接反。重插、换短线、确认 PB6→SWCLK、PB7→SWDIO
`Failed to read memory at 0xe000ed00`	DP 通、AP 不通。**按顺序查：① 把 SWD 速度降到 100~250 kHz（最常见）；② 接上 PB8→NRST；③ 另有调试器占着同一 SWD 线（拔掉 ST-Link）；④ 目标板供电不足**
`cmd_info failed`	探针 USB 掉了。重插探针 USB
插上探针没反应	只烧了 BL 没烧 IF；或地址烧错（BL=0x08000000，IF=0x0800C000）；或没插 PA11/PA12 那个 USB 口
没有虚拟 U 盘	烧的是通用版 IF。**原理与切换方法见 7.4**；想要 U 盘请烧 `MyDAPLink_IF_MSC.hex`
虚拟串口没 COM 号	装 CMSIS-DAP 驱动 / 换 USB 口
GDB `errno 10061`	OpenOCD 没在跑。先起 OpenOCD，再连 GDB
GDB `Target not examined yet`	目标板没被识别（同 `Failed to read memory` 那一行的原因）

> **排查口诀**：先分「探针自己的问题」还是「目标板的问题」—— 探针看 USB 设备是否 `OK`，目标板看 OpenOCD 有没有 `Examination succeed` 那行。 **顺序很重要**：遇到连不上，**先降速到 100 kHz 再试一次**，再去看接线 —— 本设计实测「1000 kHz 必挂、100 kHz 必通」，降速常常一步解决。

9. 刷坏了怎么救（串口 ISP）

探针固件若刷坏导致 **SWD 被关**，用 F103 芯片内建的**系统 Bootloader（串口 ISP，AN3155）**救：

1. **BOOT0 接高、BOOT1 接低**，然后复位；
2. USB-TTL 接线：**TX→PA10**、**RX→PA9**、GND 共地（注意收发交叉）；
3. 烧回 BL：

```
python rescue/isp_rescue.py out/MyDAPLink_BL.bin COM3
```

⚠️ 末尾的 `COM3` 要换成**你自己电脑上 USB-TTL 的 COM 口**（设备管理器 →「端口 (COM 和 LPT)」或命令行 `mode` 查看）； `out/MyDAPLink_BL.bin` 是**相对路径**，请在包根目录下执行。脚本依赖 `pyserial`（`pip install pyserial`）。

4. **BOOT0 接回 GND**，重新上电，再按第 3 节补烧 IF。

> 串口参数必须是 **115200, 8E1（偶校验）**——这是 STM32 ISP 的固定要求，脚本已内置； 用别的工具时也别改成 8N1。

10. 固件清单与校验值

`out/` 里三个固件均为本机编译产出：

固件	烧录地址	地址范围	大小	区域 CRC32
`MyDAPLink_BL.hex`	0x08000000	0x08000000–0x0800BBFF	48128 B	**0x3E08754F**
`MyDAPLink_IF.hex`	0x0800C000	0x0800C000–0x0801FBFF	80896 B	**0x9115E7D9**
`MyDAPLink_IF_MSC.hex`	0x0800C000	0x0800C000–0x0801FBFF	80896 B	**0xC805F60C**

`MyDAPLink_IF` 与 `MyDAPLink_IF_MSC` **二选一**：前者通用（无虚拟盘）， 后者带**虚拟 U 盘**。为什么、以及怎么切换 —— 见 **7.4**。

固件内嵌的 **Build ID** 为 `mydaplink-v1.0-g9064ad6`（本版），由 `src/upstream` 的 git tag 生成，重编译会随你自己的 tag / commit 变化； 在虚拟 U 盘的 `details.txt` 里能看到这一行。

自己核对某个 hex（看地址范围 / 向量表 / CRC）：

```
python tools/hex_inspect.py out/MyDAPLink_BL.hex out/MyDAPLink_IF.hex
```

⚠️ 这两处都是**相对路径**，请在**包根目录**下执行。只需系统里有 Python 3， **不需要装任何第三方库。**

地址范围吻合 **向量表合法** **已上板验证** **实际烧录 LEDTEST 通过**

11. 它能烧哪些芯片？为什么有的 ARM 不支持

> **核心认知**：DAPLink 固件本身**「不认识」任何芯片** —— 它只做一件事：把上位机 （OpenOCD / pyOCD / Keil）发来的 SWD / JTAG 数据包搬到目标的调试口。 所以「能烧什么」不是探针单方面决定的。

11.1 三条使用路径，覆盖范围完全不同

使用方式	谁决定「能烧什么」	覆盖范围
CMSIS-DAP 探针 OpenOCD / pyOCD / Keil / IAR	上位机的器件库	**几乎任何带 SWD / JTAG 的 ARM 芯片**
拖拽 U 盘（MSC）	固件里**内嵌的那一份** flash 算法	**只有 STM32F103**（原理见 7.4）
重新编译 IF	换掉内嵌算法	本包已预置 10 种目标（见 11.5）

11.2 原生支持的系列

档位	内核	典型芯片
原生支持 SWD 直连	Cortex-M0 / M0+	STM32F0·G0·L0、GD32E23x、nRF51、RP2040
	Cortex-M3	STM32F1·F2·L1、GD32F1·F3、LPC17xx、Kinetis K

档位	内核	典型芯片
	Cortex-M4	STM32F3·F4·L4·G4、GD32F4、nRF52、SAM D5x
	Cortex-M7	STM32F7·H7、i.MX RT、SAM V7x / S7x
	Cortex-M23 / M33 含 TrustZone	STM32L5·U5、nRF91、RA4、M2351
	Cortex-M35P / M55 / M85	STM32N6、Alif Ensemble
有条件支持	Cortex-R4 / R5 / R7 / R8	TMS570 等 —— 有 CoreSight，多走 JTAG，OpenOCD 支持部分型号
	Cortex-A5 / A7 / A9 / A53…	有 CoreSight，但多核选核 + MMU，pyOCD 不支持、OpenOCD 覆盖有限
不支持	ARM7TDMI / ARM9 / ARM11	LPC21xx、AT91SAM7 / 9、S3C2440（原因见 11.3 ①）
	非 ARM	AVR、8051、PIC、MSP430、RISC-V、PowerPC（MPC57xx）

11.3 为什么有的 ARM 不支持：三道门

```
目标芯片
  │
  ├─ ① 有 CoreSight DAP 吗？ ──否──▶ ✗ 用不了（ARM7 / 9 / 11）
  │          │ 是
  ├─ ② 调试口没被锁死吗？ ────否──▶ ✗ 用不了（引脚熔断 / RDP Level 2）
  │          │ 是
  ├─ ③ 上位机有器件配置吗？ ──否──▶ △ 能调试，但不能烧 flash
  │          │ 是
  └──────────▶ ✅ 烧录 + 调试都行
```

① 没有 CoreSight DAP —— 最根本的分水岭

Cortex 全系（M / R / A）内核对自带 CoreSight 调试访问端口，SWD 和 JTAG-DP 都从这个「总入口」进出，所以一条线就能通吃。

而 ARM7TDMI / ARM9 / ARM11（ARMv4 / v5 / v6 那一代）**那时还没有 CoreSight**： 它们要接进 CoreSight 系统，必须额外挂一颗 `ARMJTAG-DP` 再加一个 **JTAG-AP（JTAG Access Port）** —— 也就是说，它们不是「带 DAP 的核」， 而是「挂在 DAP 后面的一串 JTAG 端口」。

DAPLink 只实现了常规的 DPACC / APACC 路径 （AHB-AP / APB-AP），既没做 JTAG-AP 的扫描链，也没有 SWD 通道。

> 一句话：**「是 ARM 内核」≠「有 CoreSight」**。

② 调试口被锁死 —— 芯片自己的状态

- SWD 引脚被复用成普通 GPIO，或出厂就熔断（部分低成本 MCU 会这么干）；
- 烧到了 **RDP（读保护）Level 2** → 永久锁死，DP 直接拒绝访问；
- 芯片处于 secure 生命周期状态（Secure Debug 未解锁）。

这类情况探针再通用也连不上，**不是烧录器的问题。**

③ 上位机没有这颗芯片的器件配置 —— 最常被误判成「不支持」

> DAPLink 官方说明（daplink.io）：如果你的 IDE 里找不到要调的器件， 你**仍然可以调试**（选对 ARM CPU 架构即可），**但不能烧写 flash。**

因为「烧 flash」需要三样东西，全都写在上位机的 target 配置里 （OpenOCD 的 `target/xxx.cfg`、pyOCD 的 pack）：

需要什么	作用
memory map	哪段地址是 flash、多大
flash 算法	怎么擦、怎么编程、页大小多少
复位策略	用软复位还是硬复位

所以新出的或小众的芯片，常见情况是「探针连得上、就是没人写算法」—— **这一档不是不能用，是能力受限**：调试、读内存、跑 GDB 都行，只是写不进 flash。

11.4 怎么自己判断一颗芯片能不能烧

1. **看内核**：是 Cortex-M 吗？是 → 基本能烧（再看第 2、3 步）。
2. **查上位机**：`openocd` 的 `scripts/target/` 下有没有对应的 `.cfg`？有 → 直接能用。
3. **都没有**：退一步，只要它是 Cortex-M，先按 ARM CPU 架构连上去**调试**是可行的； 想烧 flash，需要自己补一份 target 配置 + flash 算法，或改用厂商官方工具。

举例（Infineon Traveo II `CYT4BB`，汽车 MCU）： 内核是 2× Cortex-M7 + 1× Cortex-M0+，

- ① 有 CoreSight ✅
- ② 数据手册写明 "Flash programming through SWD/JTAG interface"，调试口开放 ✅
- ③ Infineon 在 ModusToolbox 里提供了 OpenOCD 的 `infineon/cytxbb.cfg` 与 CMSIS-FLM 算法 ✅

→ **结论：能用**，把 interface 换成 `cmsis-dap.cfg` 即可。 注意它是**三核**，调试时要选核。
反过来说，这颗芯片恰恰证明：**支不支持跟品牌、跟车规无关，只看内核与协议。**

11.5 关于拖拽 U 盘的特殊说明

- U 盘**只认固件里内嵌的那一个目标系列**（本包 = STM32F103）。
- 想让 U 盘支持别的芯片，只能**重新编译 IF**（见 `README.md` 第 12 节）， 换上那个芯片的 flash 算法 —— **探针硬件不用动。**
- 本包源码里已经准备好 10 个目标工程（`src/upstream/projects.yaml`）：

工程名	目标芯片
`stm32f103xb_stm32f072rb_if`	STM32F072RB
`stm32f103xb_stm32f103rb_if`	STM32F103RB（= 本包 `MyDAPLink_IF_MSC.hex`）
`stm32f103xb_stm32f207zg_if`	STM32F207ZG
`stm32f103xb_stm32f334r8_if`	STM32F334R8
`stm32f103xb_stm32f401re_if`	STM32F401RE
`stm32f103xb_stm32f411re_if`	STM32F411RE
`stm32f103xb_stm32f429zi_if`	STM32F429ZI
`stm32f103xb_stm32f746zg_if`	STM32F746ZG
`stm32f103xb_stm32l476rg_if`	STM32L476RG
`stm32f103xb_ublox_evk_odin_w2_if`	u-blox EVK-ODIN-W2

```
./build.sh stm32f103xb_stm32f401re_if   # 举例：编出支持 F401RE 拖拽烧录的 IF
```

> **实测口径**：本包**实测通过的只有 STM32F103**（烧录 LEDTEST + `compare-sections` 6 段全 match）。上面表格里的其他系列属于**协议架构决定的能力范围**，不是逐颗芯片验证过的清单。 真要烧某颗芯片前，先确认 `openocd` 里有没有它的 `.cfg`。
